

1. Objectives:

1. To understand the concept of queue and its operations using both fixed-size and dynamic array and analyze their limitations.
2. To understand and implement linear queue, circular queue and built in queue using STL.

2. Introduction:

- **Queue:** Queue is a linear data structure that follows the First In First Out (FIFO) principle, where the element inserted first is removed first. It is widely used in scheduling, buffering, and real-time processing systems. In a queue, insertion takes place at the rear end (enqueue) and deletion takes place from the front end (dequeue). Two important variables, front and rear, are used to track the positions of elements in the queue.

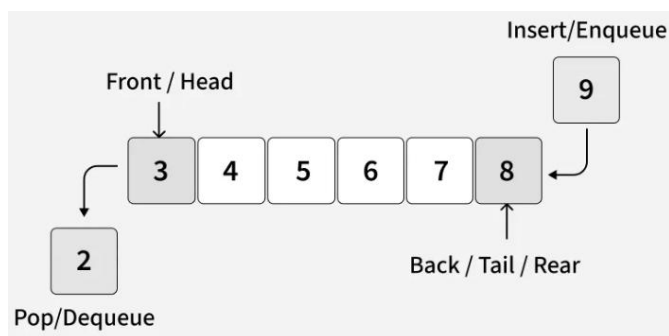


Fig 9.1: Linear Queue

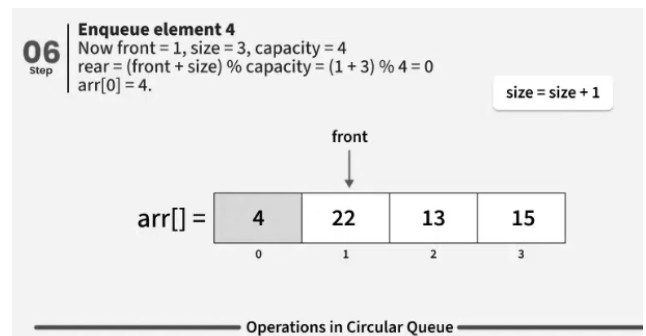


Fig 9.2: Circular Queue

- **Circular Queue:** Circular Queue is a type of queue in which the last position is connected back to the first position, forming a circle. If there are empty spaces at the beginning. A circular queue solves this problem by using a modulo (%) operation to move the rear and front pointers back to the beginning when they reach the end of the array using this $(rear + 1) \% N == front$ condition.
- **STL:** Standard Template Library (STL) in C++ is a built-in library that provides ready-made data structure and algorithms. Instead of writing everything from scratch (like array queue, stack, linked list), STL gives optimized, tested implementations like vector, stack, queue and list.

3. Procedure

Experiment 1: Linear Queue Enqueue, Dequeue, Peek

Input:

- arr[] – Array used for queue
- N – Maximum size of queue
- front – Points to first element
- rear – Points to last element
- n – Value to be inserted

Output:

- Printed queue elements OR “Queue Full/Empty” message

Algorithm:

Enqueue(arr[], N, front, rear, n):

1. **if** rear == N - 1 **then**
2. print "Queue Full"
3. return
4. **if** front == -1 **then**
5. front ← 0
6. rear ← rear + 1
7. arr[rear] ← n

Dequeue(arr[], front, rear):

1. **if** front == -1 **then**
2. print "Queue Empty"
3. return
4. front ← front + 1
5. **if** front > rear **then**
6. front ← -1
7. rear ← -1

Peek(arr[], front):

1. **if** front == -1 **then**
2. print "Queue is Empty"
3. return -1
4. return arr[front]

Display(arr[], front, rear):

1. **if** front == -1 **then**
2. print "Queue Empty"
3. return
4. **for** i ← front to rear **do**
5. print arr[i]

Experiment 2: Linear Queue using Dynamic

Input:

- arr[] – Dynamic array for queue
- N – Size of queue
- front – Points to first element
- rear – Points to last element
- n – Value to be inserted

Output:

- Printed queue elements OR “Queue Full/Empty” message

Algorithm:

SetQueue(N):

1. arr ← new array of size N

All other operations (Enqueue, Dequeue, Peek, Display) are same as Experiment 1.

Experiment 3: Circular Queue Dynamic

Input:

- arr[] – Dynamic array for queue

- N – Size of queue
- front – Points to first element
- rear – Points to last element
- n – Value to be inserted

Output:

- Printed queue elements OR “Queue Full/Empty” message

Algorithm:

SetQueue(N):

1. arr $\leftarrow$ new array of size N

IsFull(front, rear, N)

1. **if** (rear + 1) % N == front **then**
2. return true
3. **else return** false

Enqueue(arr[], N, front, rear, n)

1. **if** (rear + 1) % N == front **then**
2. print "Queue Full"
3. return
4. **if** front == -1 **then**
5. front $\leftarrow$ 0
6. rear $\leftarrow$ (rear + 1) % N
7. arr[rear] $\leftarrow$ n

Dequeue(arr[], front, rear, N)

1. **if** front == -1 **then**
2. print "Queue already Empty"
3. **return**
4. **if** front == rear **then**
5. front $\leftarrow$ -1
6. rear $\leftarrow$ -1
7. **else**
8. front $\leftarrow$ (front + 1) % N

Peek(arr[], front)

1. **if** front == -1 **then**
2. print "Queue is Empty"
3. **return** -1
4. **return** arr[front]

Display(arr[], front, rear, N)

1. **if** front == -1 **then**
2. print "Queue empty"
3. **return**
4. i $\leftarrow$ front
5. **while true do**
6. print arr[i]
7. **if** i == rear **then**
8. break
9. i $\leftarrow$ (i + 1) % N

Experiment 4: Queue using STL

Input:

- q – Queue (STL)
- Values

Output:

- Front element, updated front, and size

Algorithm:

QueueOperations(q)

1. create empty queue q
2. **push** 5 into q
3. **push** 10 into q
4. **push** 15 into q
5. **push** 20 into q
6. **push** 25 into q
7. **push** 30 into q
8. print q.front()
9. **pop** element from q
10. **pop** element from q
11. print q.front()
12. print size of q
13. End

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Linear Queue Enqueue, Dequeue, Peek

```
#include <bits/stdc++.h>
using namespace std;

#define N 5

class Queue{
public:
    int arr[N];
    int front = -1;
    int rear = -1;

    bool isFull(){
        return (rear == N - 1);
    }

    bool isEmpty(){
        return (front == -1);
    }

    void enqueue(int n){
        if (isFull()){
            cout << "Queue Full" << endl;
            return;
        }
        if (front == -1){
            front = 0;
        }
        rear++;
        arr[rear] = n;
    }
}
```

```

void dequeue(){
    if (isEmpty()){
        cout << "Queue Empty" << endl;
        return;
    }

    front = front + 1;

    if (front > rear){
        front = -1;
        rear = -1;
    }
}

int peek(){
    if (isEmpty()){
        cout << "Queue is Empty" << endl;
        return -1;
    }
    return arr[front];
}

void display(){
    if (isEmpty()){
        cout << "Queue empty" << endl;
        return;
    }

    for (int i = front; i <= rear; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}

};

int main(){
    Queue q;
    q.enqueue(5);
    q.enqueue(10);
    q.enqueue(15);

    q.display();

    return 0;
}

```

Table 9.1: Experiment 1 output

Input	Expected Output	Actual Output
	5 10 15	5 10 15

Experiment 2: Linear Queue using Dynamic

```

#include <bits/stdc++.h>
using namespace std;

class Queue{
public:
    int *arr;
    int N;
    int front = -1;
    int rear = -1;

    void setQueue(int size){
        N = size;
        arr = new int[N];
    }

    bool isFull(){
        return (rear == N - 1);
    }

    bool isEmpty(){
        return (front == -1);
    }
}

```

```

void enqueue(int n){
    if (isFull()){
        cout << "Queue Full" << endl;
        return;
    }
    if (front == -1){
        front = 0;
    }
    rear++;
    arr[rear] = n;
}

void dequeue(){
    if (isEmpty()){
        cout << "Queue Empty" << endl;
        return;
    }
    front = front + 1;
    if (front > rear){
        front = -1;
        rear = -1;
    }
}

int peek(){
    if (isEmpty()){
        cout << "Queue is Empty" << endl;
        return -1;
    }
    return arr[front];
}

void display(){
    if (isEmpty()){
        cout << "Queue empty" << endl;
        return;
    }
    for (int i = front; i <= rear; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}

};

int main(){
    cout << "Enter Size: ";
    int size;
    cin >> size;
    Queue q;
    q.setQueue(size);
    q.enqueue(5);
    q.enqueue(10);
    q.enqueue(15);
    q.enqueue(20);
    q.enqueue(25);
    q.display();
    q.dequeue();
    return 0;
}

```

Table 9.2: Experiment 2 output

Input	Expected Output	Actual Output
Enter Size: 5	Enter Size: 5 5 10 15 20 25	Enter Size: 5 5 10 15 20 25

Experiment 3: Circular Queue Dynamic

```

#include <bits/stdc++.h>
using namespace std;
class Queue{
public:
    int *arr;
    int N;
    int front = -1;
    int rear = -1;

    void setQueue(int size){

```

```

        N = size;
        arr = new int[N];
    }
    bool isFull(){
        return ((rear + 1) % N == front);
    }
    bool isEmpty(){
        return (front == -1);
    }

    void enqueue(int n){
        if (isFull()){
            cout << "Queue Full" << endl;
            return;
        }
        if (front == -1){
            front = 0;
        }
        rear = (rear + 1) % N;
        arr[rear] = n;
    }

    void dequeue(){
        if (isEmpty()){
            cout << "Queue already Empty" << endl;
            return;
        }

        if (front == rear){
            front = -1;
            rear = -1;
        }
        else{
            front = (front + 1) % N;
        }
    }

    int peek(){
        if (isEmpty()){
            cout << "Queue is Empty" << endl;
            return -1;
        }
        return arr[front];
    }

    void display(){
        if (isEmpty()){
            cout << "Queue empty" << endl;
            return;
        }
        int i = front;
        while (1){
            cout << arr[i] << " ";
            if (i == rear){
                break;
            }
            i = (i + 1) % N;
        }
        cout << endl;
    }
};

int main(){
    cout << "Enter Size: ";
    int size;
    cin >> size;
    Queue q;
    q.setQueue(size);
    q.enqueue(5);
    q.enqueue(10);
    q.enqueue(15);
    q.enqueue(20);
    q.enqueue(25);
    q.display();
    q.dequeue();
    q.dequeue();
    q.enqueue(30);
    q.display();
    return 0;
}

```

Table 9.3: Experiment 3 output

Input	Expected Output	Actual Output
Enter Size: 5	Enter Size: 5 5 10 15 20 25 15 20 25 30	Enter Size: 5 5 10 15 20 25 15 20 25 30

Experiment 4: Queue using STL

```
#include <bits/stdc++.h>
#include <queue>
using namespace std;

int main(){
    queue<int> q;
    q.push(5);
    q.push(10);
    q.push(15);
    q.push(20);
    q.push(25);
    q.push(30);
    cout << q.front() << endl;
    q.pop();
    q.pop();
    cout << q.front() << endl;
    cout << q.size() << endl;
    return 0;
}
```

Table 9.4: Experiment 4 output

Input	Expected Output	Actual Output
	5 15 4	5 15 4

6. Discussion & Conclusion:

In this lab, we learned different types of queues and understood the FIFO behavior through practical coding. Linear queue implementation helped to understand the basic working of front and rear pointers. The dynamic Circular queue by reusing empty spaces using modulo operation, making it more efficient and practical. The STL queue provided a simplified and ready-made approach, reducing implementation complexity. Some challenges were faced in managing front and rear conditions correctly, especially in circular queue where modulo operation and full condition. Overall, this lab helped to build a strong understanding of queue operations.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/queue-data-structure/> accessed at 6.25 PM on 29th April 2026.
2. <https://www.geeksforgeeks.org/dsa/introduction-to-circular-queue/> accessed at 06:35 Pm on 29th April